Ex. No.: 9
Date: 02/04/2025

DEADLOCK AVOIDANCE

Aim:

To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i
2. Find an i such that both:
 finish[i]=false and Need$_i$<= work
3. If no such i exists go to step 6
4. Compute work=work+allocationi
5. Assign finish[i] to true and go to step 2
6. If finish[i]==true for all i, then print safe sequence
7. Else print there is no safe sequence

Program Code:

```
# include <stdio.h>
# define MAX 5
# define RESOURCE_TYPES 3
void findSafe(int p, int r, int alloc[MAX][RESOURCE_TYPES],
        int avail[RESOURCE_TYPES]) {
  int work[RESOURCE_TYPES];
  int finish[MAX]={0};
  int safeSeq[MAX];
  int count=0;
for(int i=0; i<r; i++) {
    work[i] = avail[i]; }
while(count<p) {
    int found = 0;
    for(int i=0; i<p; i++) {
```

```
if (!finish[i]) {
    int canExec = 1;
    for (int j=0; j<r; j++) {
        if (max[i][j] - alloc[i][j] > work[j]) {
            canExec = 0;
            break; }
    }
    if (canExec) {
        for (int j=0; j<r; j++)
            work[j] += alloc[i][j];
    }
    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
    break; }
}
    if (!found) {
        printf ("No safe sequence exists\n");
        return; } }

printf ("Safe Sequence: ")
for (int i=0; i<p; i++) {
printf ("%d", SafeSeq[i]);
 if (i != p-1) { printf ("->"); } }
printf ("\n");
```

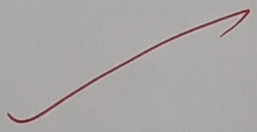
```
int main() {
    int p, r;
    printf("Enter the no. of processes:");
    scanf("%d", &p);
    printf("Enter no. of resource types:");
    scanf("%d", &r);
    int alloc[p][r], max[p][r], avail[r];
    printf("Enter allocation matrix:\n");
    for(int i=0; i<p; i++) {
        for(int j=0; j<r; j++) {
            printf("Allocation for Process P%d, Resource R%d
                    ", i+1, j+1);
            scanf("%d", &alloc[i][j]); } }
    printf("Enter maximum matrix:\n");
    for(int i=0; i<p; i++) {
        for(int j=0; j<r; j++) {
            printf("Max need for process P%d, R%d:
                    i+1, j+1);
            scanf("%d", &max[i][j]); } }
    printf("Enter available resources:\n");
    for(int i=0; i<r; i++) {
        printf("Available resource R%d:", i+1);
        scanf("%d", &avail[i]);
    }
    findsafe(p, r, alloc, max, avail);
    return 0;
}
```

OUTPUT:

Safe sequence : P1 → P4 → P2 → P3

Sample Output:

The SAFE Sequence is
P1 -> P3 -> P4 -> P0 -> P2



Result:

Thus the code to find out a safe sequence using Banker's algorithm for deadlock avoidance has been executed successfully